

Distributed Systems - 15_10

Broadcast

Now, we have multiple parties interested in communicating. Specifically, we have a source, which wants to send a message to everyone.

If there are no failures, we have a simple for loop on all processes.

When we have crashes, it's a bit more complicated.

We'll have that if a correct receives a message, all the other correct processes receive it.

Let's now focus on the for loop broadcast.

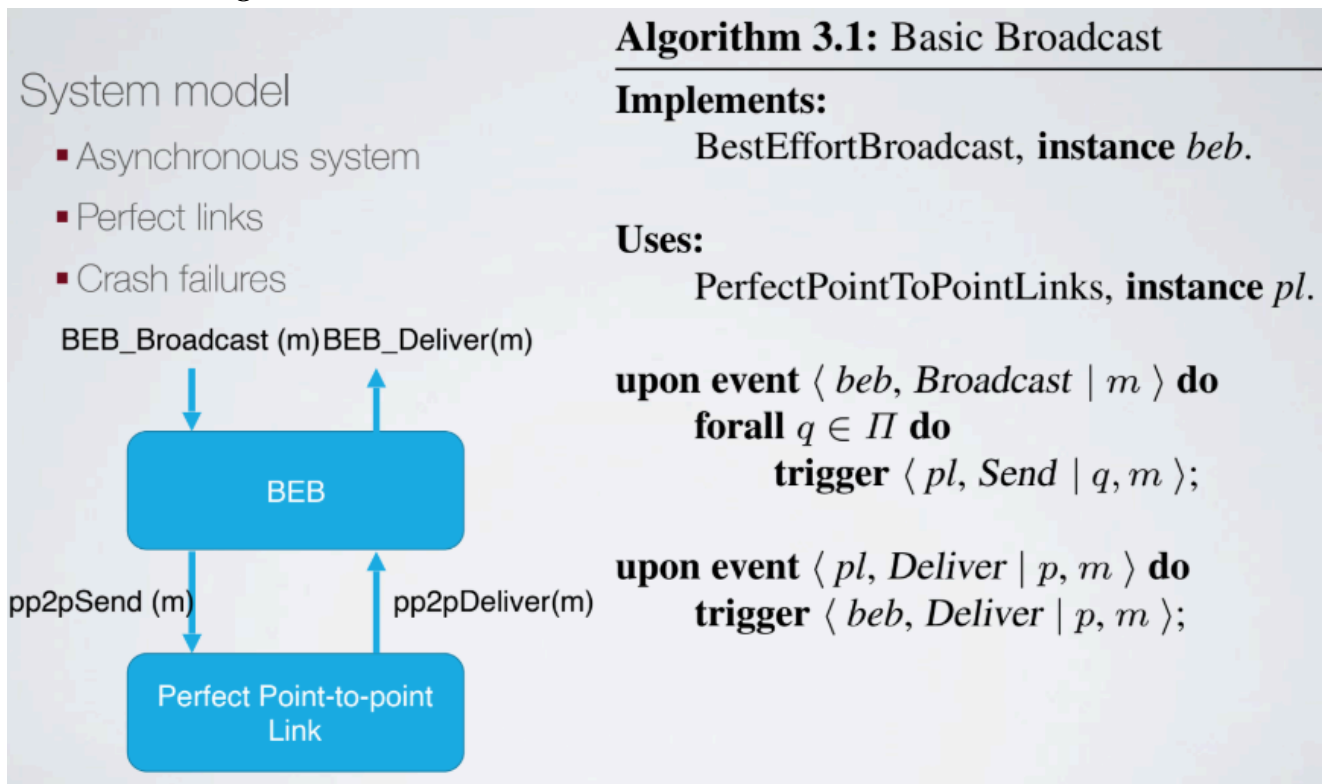
It has two events:

- Broadcast(msg)
- Deliver(msg): delivers m from p

What are the properties?

- (Validity) If a correct process broadcast a message m , then every correct process eventually delivers m .
- (No duplication) No message is delivered more than once.
- (No creation) If a process delivers a message m from process s , then m was previously broadcasted from process s .

What is the algorithm?



Let's see the proof:

- no duplication follows the no dup property of the p2p
- validity, comes from reliable delivery property of p2p, plus the fact that the sender sends the message to every other process in the system
- no creation, comes from p2p

Let us do some observations. If the sender is correct, then every process will receive the message. Let's discuss the difference from receiving the message from the link, and the delivery to the upper layer. In a broadcast, the delivery is about receiving the message from the link, but also delivering the message to the upper layer. These are different events, we may have some wait between these two (because of computations, other events we should wait for).

So, when we say deliver, we are indicating the BEB_deliver.

Also, when the sender is faulty, we can have whatever. We could have that the set of receiving processes is arbitrary, so only a subset of Π could receive the messages.

We'll have to implement a mechanism that somehow manages this thing.

So, we want a primitive with an agreement on the set of messages to be delivered, at least among the correct processes.

This primitive is named (Regular) Reliable Broadcast.

Module 3.2: Interface and properties of (regular) reliable broadcast

Module:

Name: ReliableBroadcast, **instance** *rb*.

Events:

Request: $\langle rb, \text{Broadcast} \mid m \rangle$: Broadcasts a message m to all processes.

Indication: $\langle rb, \text{Deliver} \mid p, m \rangle$: Delivers a message m broadcast by process p .

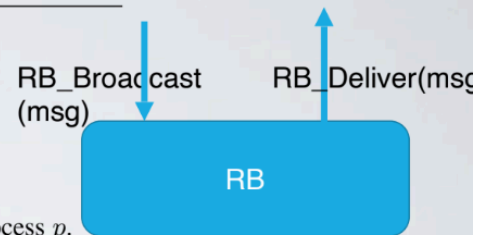
Properties:

RB1: Validity: If a correct process p broadcasts a message m , then p eventually delivers m .

RB2: No duplication: No message is delivered more than once.

RB3: No creation: If a process delivers a message m with sender s , then m was previously broadcast by process s .

RB4: Agreement: If a message m is delivered by some correct process, then m is eventually delivered by every correct process.



Same as BEB



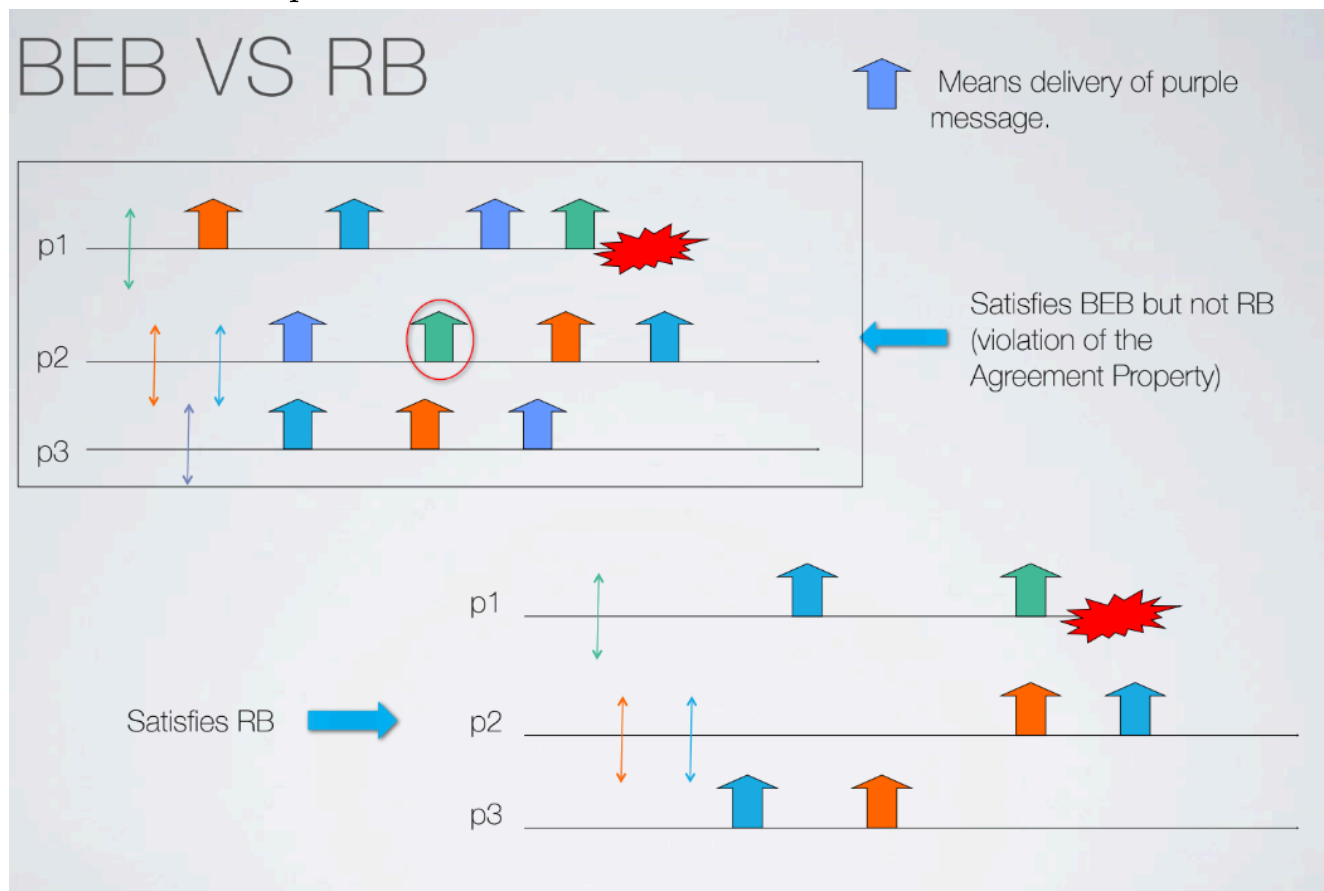
Agreement

The first three properties are similar to the best effort broadcast.

We also have a fourth property, by which if there is a single correct process that receives m , then all correct processes will receive m .

Also, the validity is a bit different, in fact it says that if I'm correct, and I do a message with broadcast $\uparrow$, then the process itself eventually delivers it's own message.

Let's see an example:



Is the first execution BEB or RB? It's BEB, we are breaking Agreement because, in infinity, the green arrow never appears in p_3 .

The second one is reliable, because the set of messages delivered by the process is the same. Also, p_2 satisfies validity by sending the message itself.

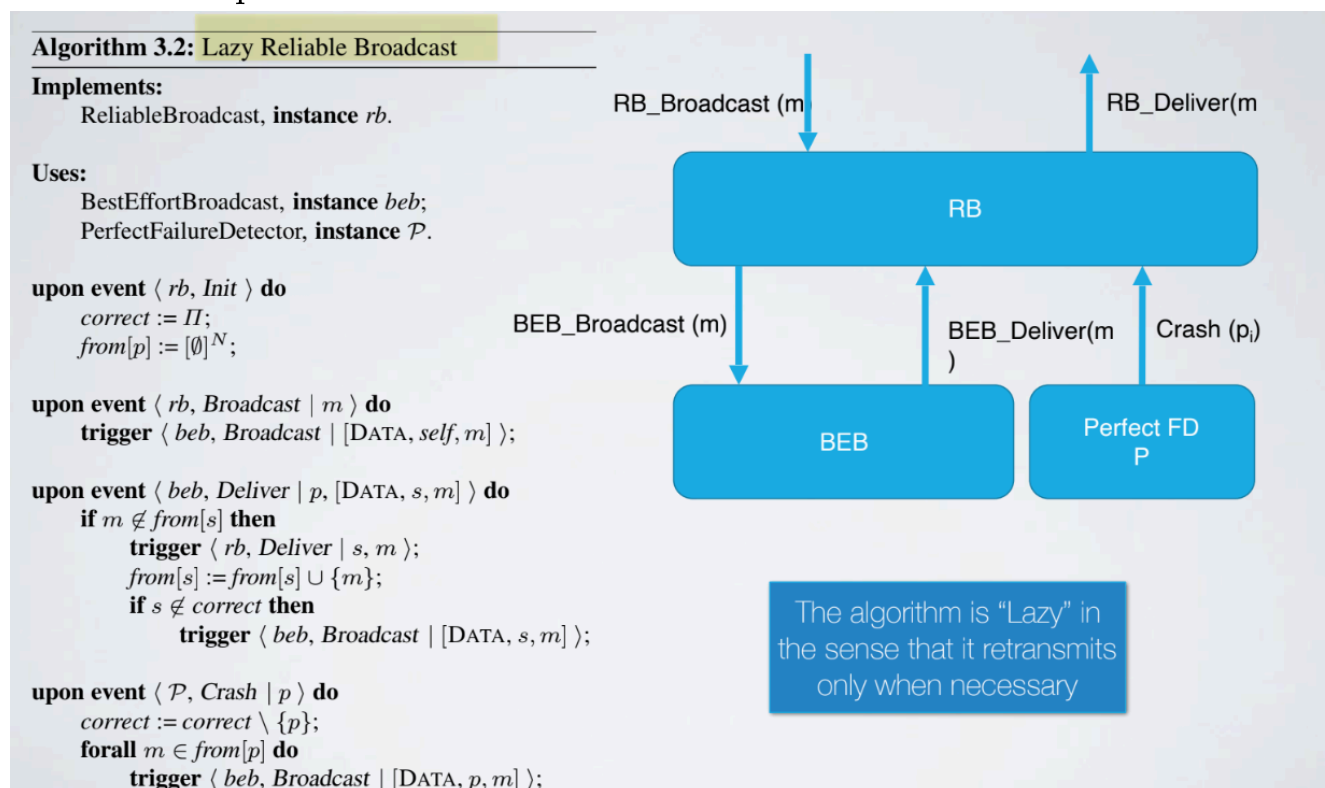
Also, $RB \implies BEB$, but $BEB \not\implies RB$

When asked, at the exam what primitive is it, we have to say the strongest one.

Let's now suppose that we cancel the red arrow in p_2 in the second example. For sure it's not RB, but not even a best effort, because we are violating the validity of the best effort.

If we have purple $\uparrow$ in p_1 (that dies), and p_3 only receives the message, it's best effort, because p_1 is not correct anymore. If p_3 does not receive it, it's reliable.

How do we implement now the RB?



At the start we have the set of correct processes equal to Π , but also $from[p] = [\emptyset]^N$. This means that, for every process in the system, I'm creating an empty set (it's like a dictionary in Python).

Let's now see the Broadcast handler. We trigger a Broadcast in beb with a message made by three elements, we have m (content of the message), $self$ as my id, DATA is a flag.

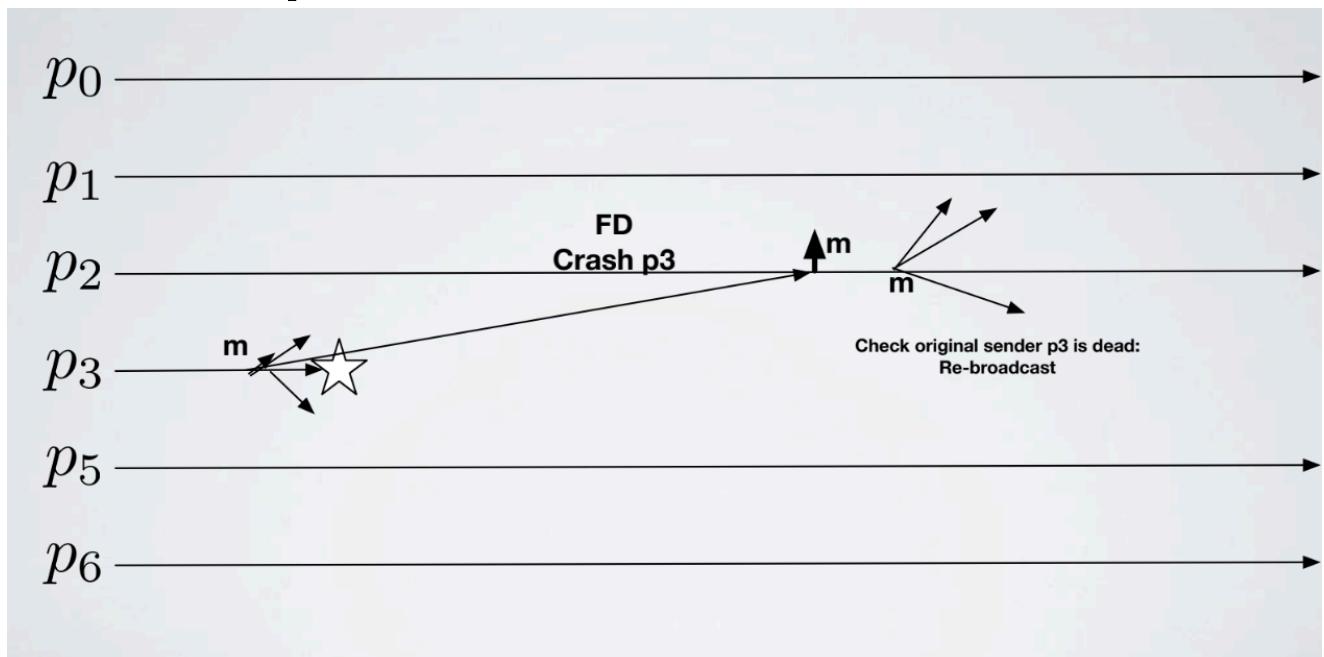
When another process receives a broadcast message, we have the delivery in beb. At first, we'll have that p and s is equal (the sender is the same one as $self$). We then check, for no duplication, if the message is already in $from[s]$. If not, we

trigger the delivery, we update $from[s]$ to include the message m , and then I check the source of the message. If it's not correct, we broadcast the message again.

What if p_1 broadcasts a message and dies? We'll know for P that p_1 crashed, so when P detects this we take a look at all the messages from p_1 in $from[p_1]$ and broadcast the messages again (using $self$ as the original Id, so not p_2 let's say but p_1).

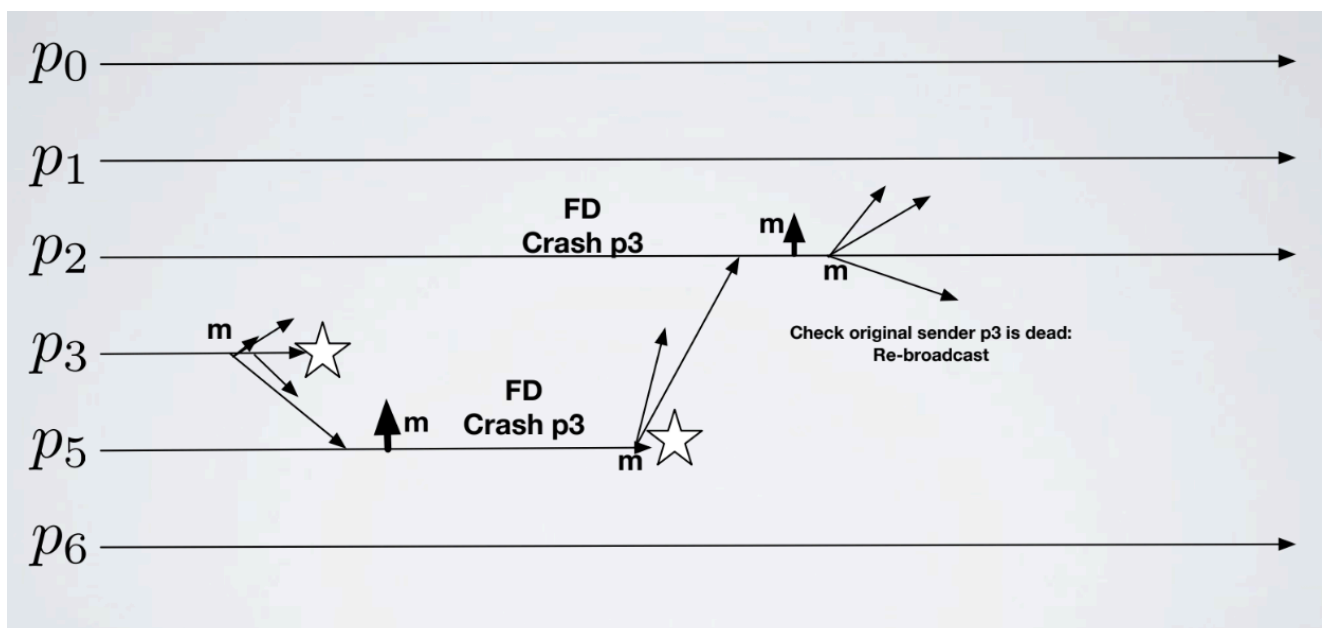
So the source is necessary for this reason, because the message could be a "relay" of a message (and so we don't want to take the id of the sender, but rather the source in s).

Let's see an example:



Because p_3 is not correct, we broadcast the message again (from p_2).

Another one:

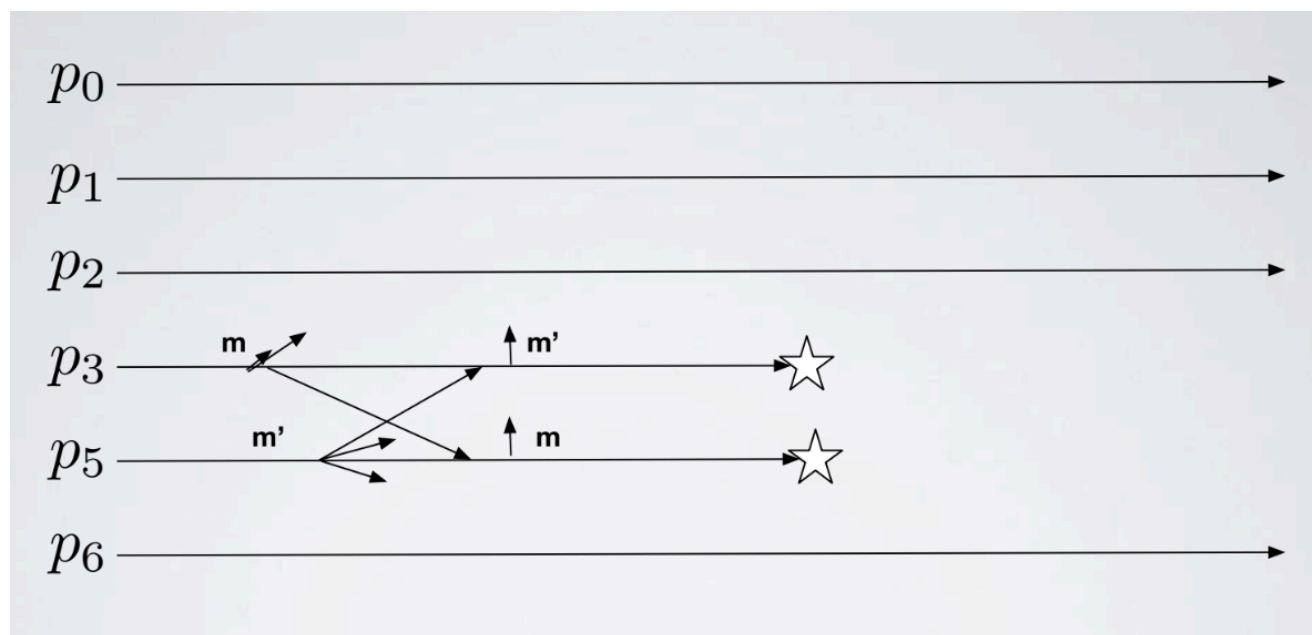


From p_5 we see that p_3 has died (with Crash handler), and thus we broadcast the

messages again. If we crash while sending the messages, there would be p_2 that completes the job.

If i receive a message from someone who's dead, we send the message again.

Let's see another:



Here we don't have a problem, because the messages arrives at a process that crashes.

Now, let's see the formal proof.

Validity and no creation are ensured by the BEB.

No duplication is ensured by if $m \notin \text{from}[s]$.

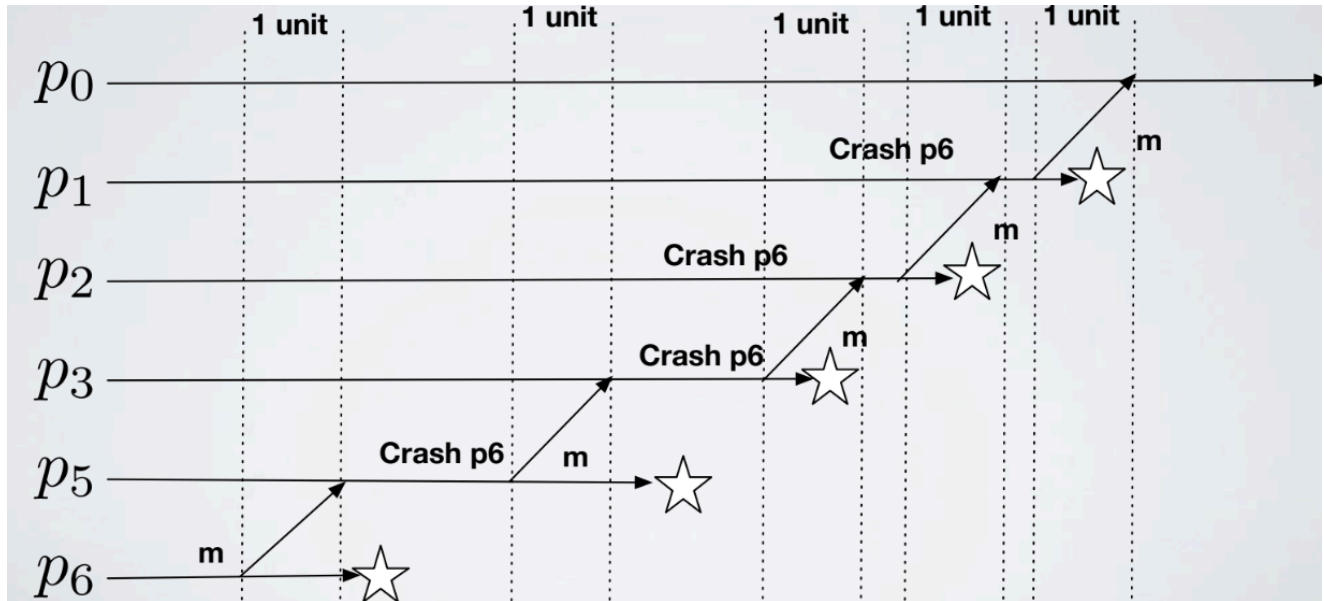
The agreement: suppose by contradiction that a process p is correct and delivers a message m with sender q , while another correct p' does not deliver it:

- if q does not crash, then by BEB also p' also delivers m
- if q crashes and p detect the crash before receiving the message, then p relays the message
- *slide*

What is the message complexity? In the best case, we send N messages (to N processes), this supposing the process remains correct. In the worst case, N^2 , because every process (N processes) send N messages, so $N \times N = N^2$. This phenomenon is known as "broadcast storm".

What about the message delays (communication steps)? In the best case, we only have 1 time unit to send the broadcast and receive it. In the worst case, $N - 1$, because every process that receives the message dies, and we have to send once

again the message. Here a visual representation:



What happens if, instead of having P , we have $\Diamond P$? Are the properties violated? If $\Diamond P$ tells that p_1 has crashed and it actually crashed, then it's not a problem. What happens if p_1 is alive and we detect it as dead? We will broadcast the messages thinking that p_1 has died, but this is not a problem because the messages could be sent two times but they will be delivered only once.

What happens in a fail-silent situation? We use an algorithm named Eager Reliable Broadcast.

Algorithm 3.3: Eager Reliable Broadcast

Implements:

ReliableBroadcast, **instance** rb .

Uses:

BestEffortBroadcast, **instance** beb .

upon event $\langle rb, Init \rangle$ **do**

$delivered := \emptyset$;

upon event $\langle rb, Broadcast \mid m \rangle$ **do**

trigger $\langle beb, Broadcast \mid [DATA, self, m] \rangle$;

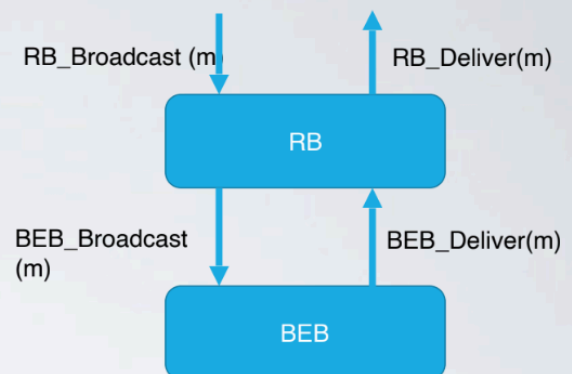
upon event $\langle beb, Deliver \mid p, [DATA, s, m] \rangle$ **do**

if $m \notin delivered$ **then**

$delivered := delivered \cup \{m\}$;

trigger $\langle rb, Deliver \mid s, m \rangle$;

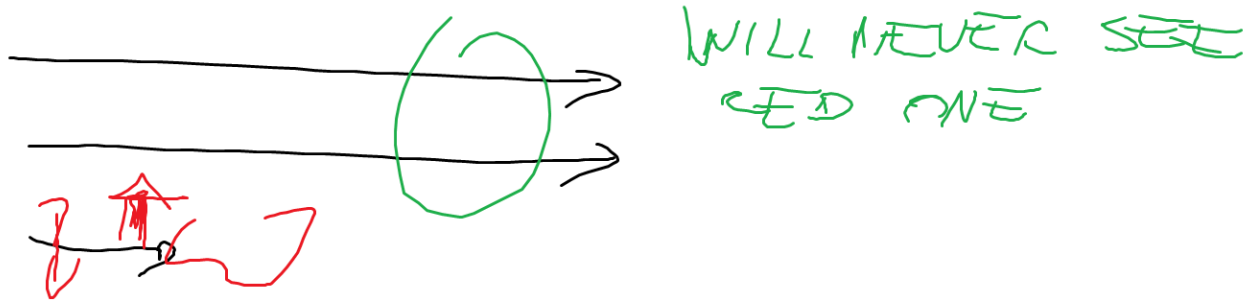
trigger $\langle beb, Broadcast \mid [DATA, s, m] \rangle$;



The algorithm is Eager in the sense that it retransmits every message

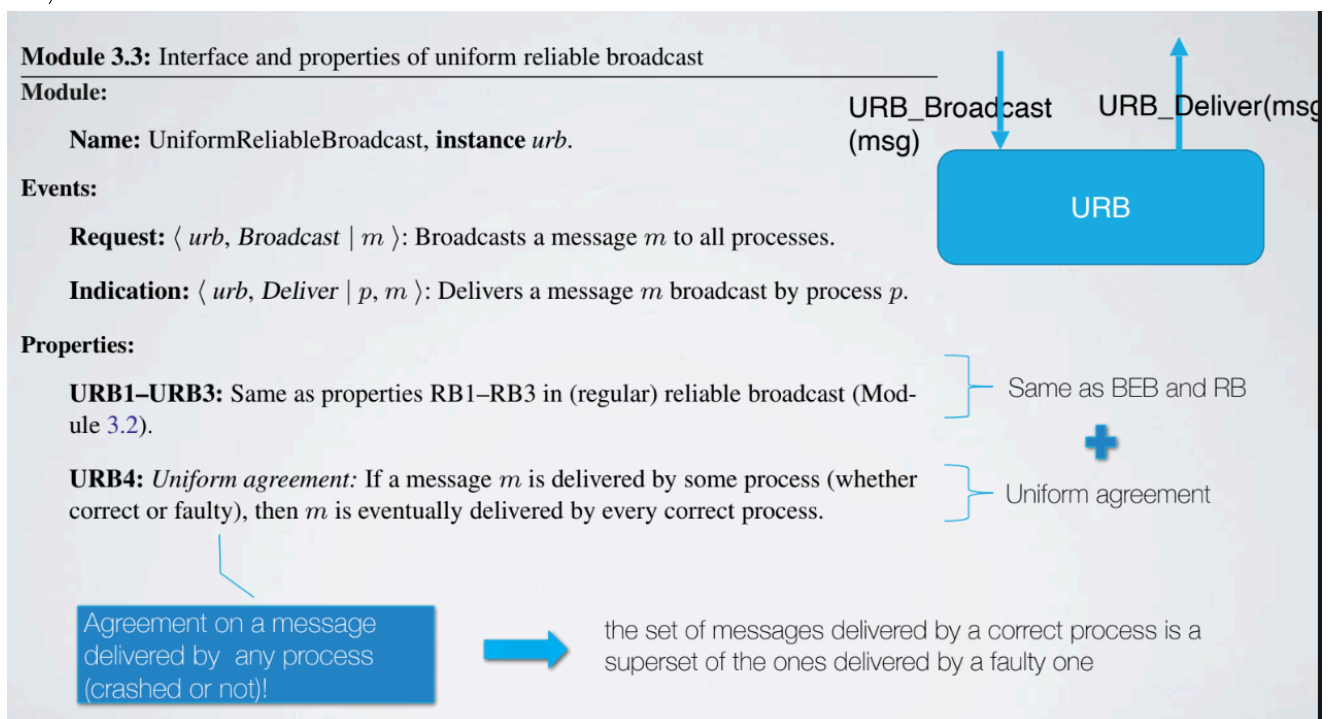
In the best case, we create N^2 messages. What is the message delay of this algorithm? In the best case it's 1, in the worst N .

What we don't like about the reliable broadcast?



So, what we want, even if a process broadcasts a message and it's faulty, the correct delivers it anyways.

So, we take a look at the Uniform Reliable Broadcast.



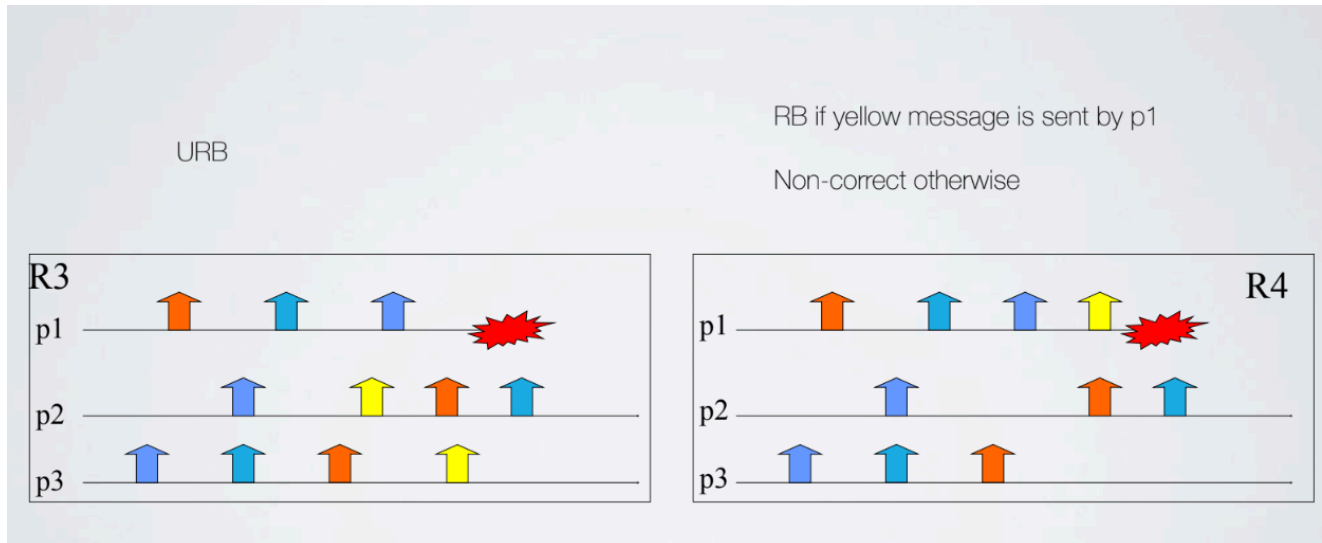
With the fourth process, we ensure that every process receives a message coming even from faulty ones.

Also, in all the broadcasts, we could have two processes that are correct and deliver the same set of messages in different order. This is fine for now, we'll see in the future how to agree in the order.

The last property implies that a correct process has to deliver more than the faulty one.

In distributed computing, uniform may mean that it's a property that also involves the faulty processes. It may also mean an algorithm that does not need to know the number of processes in the system.

Let's see an example:



R_3 is URB because the messages of p_2 and p_3 are a superset of the messages sent by p_1 (faulty).

The second (R_4) is reliable if the yellow message is sent by p_1 , if for example is sent by p_2 then it's not a correct broadcast.

How do we implement this strong primitive?

Let's say that p_1 wants to broadcast a message. Our algorithm will have, first, to send the message to everyone. At a certain point, I will receive the message from myself. At that point, can I create the delivery arrow to the upper layer? No, because I have to wait for other processes to deliver the message. So, I have to wait, and to do so we store the message in a $\text{Pending} = \{M\}$. When p_3 receives the message, he sends an ACK. p_1 , when he receives an ACK, will know that two processes has received the message. We still cannot create the arrow, because p_1 and p_3 might die, and p_2 could never know about the message. We have to wait for ACKs coming from everyone. At some moment, we can deliver. We can only deliver when the set of processes from who we received the ack, is equal to $\Pi/\text{crashed}$.

We could, also, substitute the ACK with the broadcasts. If a process broadcast again a message, and the original message receives it, we use that thing as an ACK, removing the need for them.

Algorithm 3.4: All-Ack Uniform Reliable Broadcast

Implements:
 UniformReliableBroadcast, **instance** *urb*.

Uses:
 BestEffortBroadcast, **instance** *beb*.
 PerfectFailureDetector, **instance** $\mathcal{P}$.

```

upon event  $\langle \mathcal{P}, \text{Crash} \mid p \rangle$  do
     $correct := correct \setminus \{p\}$ ;

function candeliver(m) returns Boolean is
    return ( $correct \subseteq ack[m]$ );

upon exists (s, m)  $\in pending$  such that candeliver(m)  $\wedge m \notin delivered$  do
     $delivered := delivered \cup \{m\}$ ;
    trigger  $\langle urb, Deliver \mid s, m \rangle$ ;

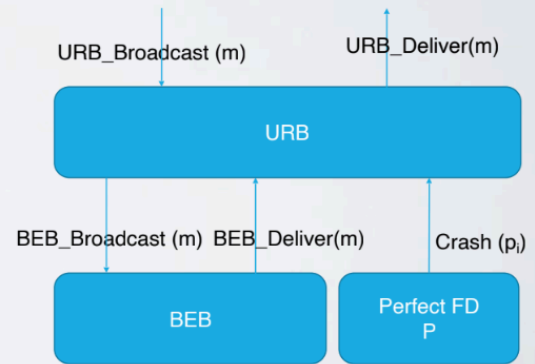
upon event  $\langle urb, Init \rangle$  do
     $delivered := \emptyset$ ;
     $pending := \emptyset$ ;
     $correct := \Pi$ ;
    forall m do  $ack[m] := \emptyset$ ;

upon event  $\langle urb, Broadcast \mid m \rangle$  do
     $pending := pending \cup \{(self, m)\}$ ;
    trigger  $\langle beb, Broadcast \mid [DATA, self, m] \rangle$ ;

upon event  $\langle beb, Deliver \mid p, [DATA, s, m] \rangle$  do
     $ack[m] := ack[m] \cup \{p\}$ ;
    if (s, m)  $\notin pending$  then
         $pending := pending \cup \{(s, m)\}$ ;
        trigger  $\langle beb, Broadcast \mid [DATA, s, m] \rangle$ ;
        
```

```

sequenceDiagram
    participant URB
    participant BEB
    participant P as Perfect FD P
    URB->>BEB: URB_Broadcast (m)
    BEB->>URB: BEB_Deliver (m)
    BEB->>P: Crash (p)
    P->>URB: URB_Deliver (m)
    
```



We also have a function used for an handler. The handler is triggered when we receive an ACK for everyone.

This generated N^2 messages, even in the best case.